UMHACKATHON 2026 · DOMAIN 1 · AI SYSTEMS & AGENTIC WORKFLOW AUTOMATION

Software Architecture.

The shipped system: surfaces, components, runtime data flows, and the audit substrate behind every GLM decision.

TEAM · OPUS MAGNUM

Oybek Odilov
Al-Bazeli Gameel Abduljalil
Lee Yih Shen
Samandar Erkinjonov
Abdugaffor Odilov

ONBOARD

POWERED BY Z.AI GLM · REGULATED ONBOARDING ENGINE

SAD · SOFTWARE ARCHITECTURE APRIL 2026 SUBMISSION RELEASE

1. System Overview

Opus Magnum is a three-surface, single-engine platform:

- **Admin surface** (`/admin/*`) — templates authored in a UI, with GLM in the loop.
- **Owner surface** (`/onboard/*`) — runs a city's template step by step; watches GLM extract, verify, cross-check, and propose fixes for every document.
- **Visitor surface** (`/buildings` , `/buildings/[id]`) — browses published buildings in 3D, asks a grounded assistant.

Under the surfaces is one **adaptive onboarding engine**: a LangGraph `StateGraph` compiled from a JSON template, with GLM invoked at every reasoning node and the full decision trail persisted for audit.

2. Component Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      Frontend (Next.js 16, TS)                           │
│                                                                          │
│   Admin UI            Owner UI                Visitor UI                 │
│   /admin/*            /onboard/[id]           /buildings/[id]            │
│   • cities & templates  • workflow canvas     • 3D scene (R3F + GLB)     │
│   • template assistant  • step form / upload  • grounded assistant       │
│                         • autofill confirm    • camera-nav cues          │
│                         • decision log drawer                            │
│                         • rewind controls                                │
└──────────────────────────────────────────────────────────────────────────┘
          │                      │                      │
          │            HTTP + SSE (CORS)                │
          ▼                      ▼                      ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                  Backend  (FastAPI, Python 3.12)                         │
│                                                                          │
│   /cities       /templates       /buildings    {id}/assistant            │
│                                                                          │
│    ┌──────────────────────────────────────────────────────────────────┐  │
│    │              GLM Reasoning Core (ILMU API)                       │  │
│    │   OpenAI SDK → https://api.ilmu.ai/v1  (ilmu-glm-5.1)            │  │
│    │                                                                  │  │
│    │   Eight tools exposed to GLM:                                    │  │
│    │    draft_template, explain_field, extract_document,              │  │
│    │    verify_against_criteria, cross_check_documents,               │  │
│    │    propose_auto_fix, summarize_for_review,                       │  │
│    │    answer_visitor_question                                       │  │
│    │                                                                  │  │
│    │   Every call → Decision → step_runs.decision_log                 │  │
│    └──────────────────────────────────────────────────────────────────┘  │
│                                                                          │
│    ┌──────────────────────────┐      ┌──────────────────────────────┐    │
│    │   LangGraph Runtime      │◄────►│    Template Compiler         │    │
│    │   StateGraph per run     │      │    JSON steps → nodes+edges  │    │
│    │   SQLite/Postgres        │      │                              │    │
│    │   checkpointer           │      │    Six-primitive registry    │    │
│    │   thread_id = run/rev    │      │                              │    │
│    └──────────────────────────┘      └──────────────────────────────┘    │
│                                                                          │
│    ┌──────────────────────────┐      ┌──────────────────────────────┐    │
│    │  Doc Processing Hub      │      │  Building Profile Builder    │    │
│    │  pdf → text (pypdf +     │      │  content-doc extractions     │    │
│    │  OCR fallback)           │      │   → profile + scene_config   │    │
│    │  image → OCR             │      │                              │    │
│    └──────────────────────────┘      └──────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────────┘
          │
```

```
          ▼
    ┌──────────────────────────────────────────────────────────────────┐
    │  Postgres (prod, Render) / SQLite (dev, var/dev.sqlite)          │
    │  users · cities · workflow_templates · buildings ·               │
    │  workflow_runs · step_runs · documents                           │
    │                                                                  │
    │  Object store: local filesystem (var/uploads) or S3 (prod path)  │
    └──────────────────────────────────────────────────────────────────┘
```

3. Runtime Data Flow

3.1 Admin — authoring a template

1. Admin enters a one-line description in the template editor.
2. `POST /templates/{id}/draft-with-ai` invokes `GLMClient.call_tool` with the `draft_template` function schema and the `DRAFT_TEMPLATE_SYSTEM` system prompt.
3. GLM returns a structured `steps: [...]` array conforming to the six allowed primitives.
4. Admin tweaks fields in the editor (drag-reorder, primitive-specific config, criteria list).
5. `POST /templates/{id}/publish` flips `status='published'`.
6. `POST /templates/{id}/assistant` streams tutor-style answers (via the `explain_field` tool) over SSE when the admin asks a question about any step.

3.2 Owner — running a workflow

1. Owner creates a building (`POST /buildings`) and starts the run (`POST /buildings/{id}/start`), which creates a `workflow_runs` row and a fresh checkpoint thread.
2. The runner:
3. loads `workflow_templates.steps` ,
4. compiles them into a `StateGraph` via `compile_template(steps)` ,
5. opens the SQLite/Postgres checkpointer (`thread_id = "{run_id}:{rev}"`),
6. invokes the graph with the current `RunState` .
7. Each primitive node inspects `state.pending_input` ; if nothing is queued, it returns `awaiting_user=True` and the conditional edge `_route_after_step` routes to `END` , pausing the graph.
8. The API surfaces the paused state: `GET /buildings/{id}/run` returns `RunState` ; `GET /buildings/{id}/run/graph` returns a canvas-friendly `{nodes, edges}` shape.
9. **Form submit / file upload:**

10. `POST /buildings/{id}/run/steps/{step_id}/submit` merges form values into `pending_input[step_id]` and ticks the runner.
11. `POST /buildings/{id}/run/steps/{step_id}/upload` streams multipart files through `processing/pipeline.py` (text extraction via `pypdf` or OCR), persists `Document` rows, then ticks the runner.
12. **Autofill loop:**
13. `POST /buildings/{id}/run/steps/{step_id}/autofill` runs `extract_document` over the uploaded doc and, if a regulator-issued document exists, runs `propose_auto_fix` to generate corrections for the form's draft values.
14. The owner sees proposed values inline, accepts or overrides, and submits.
15. Every GLM call inside a primitive passes through a `Decision` sink that tags each entry with `step_id` and appends it to `state.decision_log`. The runner mirrors these into `step_runs.decision_log` when it persists the snapshot.
16. **Rewind:** `POST /buildings/{id}/run/rewind/{step_id}` increments `RunState.rev`, allocates a new checkpoint thread (`thread_id = "{run_id}:{rev+1}"`), and restarts the graph from the chosen step. Earlier `step_runs` are preserved.
17. When the `publish` primitive fires, the runner calls `build_profile(state)` and writes `profile` + `scene_config` onto the `Building` row; `published_at` stamps.

3.3 Visitor — asking a grounded question

1. Visitor opens a published building; the frontend fetches `GET /buildings/{id}`.
2. Chat sends `POST /{id}/assistant` with the message history.
3. The backend assembles grounding material: `building.profile`, `building.scene_config`, and content-doc extracts (budgeted to ≤ 60k chars).
4. `GLMClient.call_tool` invokes `answer_visitor_question` (or, for free-form chat, `complete(stream=True)` with the `ANSWER_VISITOR_QUESTION_SYSTEM` prompt) and streams tokens back via `text/event-stream`.
5. Scene-navigation cues in the response (e.g., "look at the rooftop") are interpreted by the frontend as camera moves, not as document requests.

4. Data Model

TABLE	PURPOSE	KEY COLUMNS
`users`	seeded admin + owner (mock auth)	`id`, `email`, `name`, `role`
`cities`	jurisdiction keyed by name	`id`, `name`, `country`, `region`, `created_by_admin_id`
`workflow_templates`	admin-authored JSON template	`id`, `city_id`, `name`, `version`, `steps (JSON)`, `status`
`buildings`	onboarding + publication record	`id`, `owner_id`, `city_id`, `name`, `address`, `coordinates (JSON)`, `status`, `profile (JSON)`, `scene_config (JSON)`, `published_at`
`workflow_runs`	one per building onboarding	`id`, `building_id`, `template_id`, `state (JSON)`, `status`, `current_step_id`
`step_runs`	per-step audit trail	`id`, `run_id`, `step_id`, `primitive`, `input (JSON)`, `output (JSON)`, `decision_log (JSON)`, `status`, `started_at`, `completed_at`
`documents`	uploaded files + extractions	`id`, `building_id`, `step_run_id`, `doc_class` (`compliance` | `content`), `doc_type`, `mime_type`, `filename`, `file_url`, `extracted (JSON)`, `verification_result (JSON)`, `processing_status`

Two JSON stores that evolve independently. `workflow_templates.steps` holds the domain-specific authoring shape; `workflow_runs.state` holds the frozen `RunState` snapshot. Both are JSONB on Postgres and TEXT on SQLite — same code either way.

`decision_log` is the AI audit substrate. Every entry in `step_runs.decision_log` is one GLM interaction: call_id, tool name, request messages (with bodies > 4k chars elided and image data URIs stripped), response content + tool_calls + usage, duration. Snapshot tests assert the shape as a regression net against silent GLM behaviour drift.

5. Module Boundaries

MODULE	RESPONSIBILITY
`backend/app/glm/`	Single point of contact with the ILMU API. `GLMClient`, eight tool schemas, eight system prompts, decision-log emission. Nothing outside this module talks to OpenAI.
`backend/app/workflow/`	Template compilation, `RunState` reducers, six primitives, runner, checkpointer factory.
`backend/app/workflow/primitives/`	One file per primitive. Each exports a factory that returns the graph node. New primitives = new file + registry decorator.
`backend/app/processing/`	PDF + image parsing. Returns plain text. Never calls GLM.
`backend/app/profile/`	Pure function: `RunState` → (`profile`, `scene_config`). No side effects.
`backend/app/db/`	SQLAlchemy models, session factory, `create_all` / `drop_all` schema bootstrap.
`backend/app/api/`	FastAPI routers (`onboard`, `admin`, `buildings`, `assistant`). Thin — delegate logic to the modules above.
`backend/app/seed/`	Demo bootstrap: `seed_demo` (initial), `reseed_demo` (in-place reset), `demo_autofill` (pre-populated form values), `shah_alam_template`, four markdown demo documents.
`frontend/lib/`	Typed API client, SSE helper.
`frontend/components/onboard/`	Workflow canvas (custom DAG renderer), step nodes, step form, upload panel, decision log, rewind controls.
`frontend/components/building/`	3D scene, assistant chat.
`frontend/components/admin/`	Cities & templates UI, template assistant.

A reviewer should be able to answer, for any module, three questions — *what does it do*, *how is it used*, *what does it depend on* — from the first thirty lines of the primary file.

6. Interfaces

6.1 HTTP (JSON except where noted)

```
GET  /health                                                → liveness + active mo

# Admin
GET  /cities                                                → list cities
POST /cities                                                → create city
GET  /cities/{cityId}/templates                             → list templates for c
POST /cities/{cityId}/templates                             → create template
GET  /templates/{id}                                        → read template
PUT  /templates/{id}                                        → update steps
POST /templates/{id}/publish                                → flip to published
POST /templates/{id}/draft-with-ai                          → GLM drafts steps
POST /templates/{id}/assistant                              → SSE, explain-field t

# Owner / Onboard
POST /buildings                                             → create building
POST /buildings/{id}/start                                  → create run + tick
GET  /buildings/{id}/run                                    → RunState snapshot
GET  /buildings/{id}/run/graph                              → canvas nodes + edges
GET  /buildings/{id}/run/stream                             → SSE progress events
POST /buildings/{id}/run/steps/{stepId}/submit              → form / ack payload
POST /buildings/{id}/run/steps/{stepId}/upload              → multipart files
GET  /buildings/{id}/run/steps/{stepId}/demo-preview        → seeded demo doc prev
POST /buildings/{id}/run/steps/{stepId}/autofill            → extract + propose fi
POST /buildings/{id}/run/steps/{stepId}/edit-form           → admin edit form valu
POST /buildings/{id}/run/steps/{stepId}/edit-upload         → replace / delete upl
POST /buildings/{id}/run/auto-fix                           → run-level fix propos
POST /buildings/{id}/run/rewind/{stepId}                    → bump rev, rerun from
POST /buildings/{id}/publish                                → force publish

# Visitor
GET  /buildings                                             → published list
GET  /buildings/{id}                                        → profile + scene_conf
POST /{id}/assistant                                        → SSE grounded answer
```

6.2 GLM tool schemas (OpenAI function-calling)

Eight tools, one system prompt each. Source: `backend/app/glm/tools.py` and `backend/app/glm/prompts.py`:

`draft_template`, `explain_field`, `extract_document`, `verify_against_criteria`, `cross_check_documents`, `propose_auto_fix`, `summarize_for_review`,

`answer_visitor_question`.

7. Process Model

The backend is a single FastAPI process (uvicorn). Each HTTP request handler is stateless; persistence is in Postgres/SQLite (app data) or the LangGraph checkpointer (graph state). A graph invocation compiles fresh each tick — compile is cheap and avoids stale checkpointer references across requests.

Long-lived streams are SSE (GET for visitor / run-progress; POST-via-fetch for assistant chat and template assistant). There are no WebSockets.

8. Failure Modes & Recovery

FAILURE	DETECTED IN	RECOVERY
GLM 429 rate limit	`GLMClient._chat`	Exponential backoff with jitter, up to 4 retries
GLM 5xx	`GLMClient._chat`	Same retry loop
GLM returns invalid JSON	`complete_json` / `call_tool`	Returns `{_error, _raw}`; caller treats as failure without crashing
Primitive raises	`runner.tick`	Catches, persists `status='failed'`, returns snapshot; UI shows error badge
Verification fails (criterion unmet)	`upload_compliance` primitive	Graph pauses on same step; owner re-uploads or rewinds
Cross-check surfaces trivial diff	`cross_check_documents` primitive	Severity ≤ "minor" hidden from owner UI; logged for audit
Autofill proposes a wrong value	UI confirmation dialog	Owner overrides; original value persisted in decision log
OCR binary missing	`processing/pdf.py`, `processing/image.py`	Returns empty string; decision log marks `ocr_used=false`
Scanned PDF with no text layer	`extract_pdf_text`	Automatic `pdf2image` + `pytesseract` fallback if `[ocr]` extra installed
Owner abandons run	n/a	`RunState` is checkpointed; next `tick()` resumes from paused step
Owner needs to redo earlier steps	rewind endpoint	Bumps `RunState.rev`; new checkpoint thread; earlier `step_runs` preserved

9. Non-functional Properties

Auditability. Every GLM decision is persisted with call_id, duration, input, and output. This is the single highest-value non-functional property — the platform's value proposition rests on judged decisions being inspectable.

Resumability. Runs survive process restarts via the LangGraph checkpointer. Owner can return days later and continue.

Reversibility. Rewind is first-class: `RunState.rev` is part of the checkpoint thread id, so a re-run is a clean slate that doesn't collide with the prior thread.

Determinism of the template model. Template JSON → `StateGraph` → same edges every time. The compiler is a pure function of its input.

Deployability. A single Render Blueprint (`render.yaml`) provisions the backend service and a managed Postgres 16 instance; the frontend deploys to Vercel as a static Next.js app. No bespoke infrastructure.

10. Project Timeline & Team Collaboration

10.1 Planning artifact

Project planning lived in `tasks.md` at the repository root, in preference to a SaaS task board (Linear / Trello / Jira). The trade-off was deliberate:

- **Single source of context for AI-assisted work.** Claude Code reads in-repo markdown natively; an external board would have required MCP wiring or context-paste at every session and fragmented planning between two systems.
- **Audit trail via git.** Every change to the plan is captured by `git log` ; a SaaS board adds another silo to consult and reconcile.
- **Travels with the codebase.** Cloning the repository is sufficient to onboard a teammate or a reviewer — nothing lives behind a separate login.
- **Zero account-provisioning overhead** for a five-person student team working under a hard deadline.

The plan is structured as three waves (foundation → parallel features → polish), with eleven task IDs (T1–T11) and an explicit dispatch graph. See `tasks.md` at the repo root for the working copy.

10.2 Team & responsibilities

MEMBER	ROLE	SCOPE
Oybek Odilov	Team lead, architect, lead engineer	Full backend, GLM integration, system design, initial product framing
Abdugaffor Odilov	Frontend & documentation	UI components, document drafts, PDF rendering
Al-Bazeli Gameel Abduljalil	Pitch video	Recording, editing, demo footage
Lee Yih Shen	Orchestration & quality assurance	Work coordination, test coverage, review cycles
Samandar Erkinjonov	Product & presentation	Product narrative, pitch deck content

> ***Note on git authorship.*** *The repository's commit log appears under a single git identity (* `avinmaster` *) because Abdugaffor pairs from the team lead's secondary workstation, which carries the same Claude Code + git configuration. Authorship in git reflects machine identity, not effort split; the role table above is the accurate division of work.*

10.3 Development timeline

The preliminary-round build window ran **2026-04-24 → 2026-04-26 (UTC+8)**, captured by twenty-two commits in chronological order:

```
2026-04-24  Initial commit
2026-04-24  Initial commit                      (repo bootstrap)
2026-04-25  Finished MVP
2026-04-25  docs: add frontend redesign spec
2026-04-25  frontend: full redesign — Warp/Vercel-style dark theme + motion
2026-04-25  frontend: fix exterior 3D, dial back gradients, harden chat rendering
2026-04-25  Finish layout and redesign
2026-04-25  Change texts to the more intuitive
2026-04-25  Fix [object Object]
2026-04-25  Wire backend for Vercel + Render + Neon deploy
2026-04-25  Collapse deploy to a single Render Blueprint
2026-04-25  Skip corepack on Render — pnpm signing key verify breaks the build
2026-04-25  Drop pnpm-workspace.yaml — frontend is a single package, not a workspa
2026-04-25  Tighten admin assistant prompts to emit clean block markdown
2026-04-25  Correct submitting moments
2026-04-25  Add AI_MODEL switch to route between GLM and OpenAI
2026-04-25  Fall back to index when StepForm field name is missing
2026-04-26  Add reseed script that drops/recreates tables instead of nuking the SQ
2026-04-26  Fix onboarding
2026-04-26  Finish fixes on prompts
2026-04-26  Rewrite submit docs
2026-04-26  Finish submit docs
```

The cadence is intentionally compressed: the team built on top of a pre-architected plan held in `tasks.md`, with integration risk front-loaded into Wave 0 (T1–T4) and parallel feature work driving the bulk of day two. Wave 2 polish (deploy hardening, onboarding fixes, prompt tightening, doc rendering) runs through the final day.

See the TAD for technology-selection rationale and the QATD for the test strategy.